Kendr Optimizer

Verification-Gated Typed Token Reduction for Provider-Neutral LLM Contexts

9 native engines	7 audited harnesses	2 exact BPE profiles

A deterministic, fail-open framework for structure-aware compaction, byte-exact reconstruction, cache protection, and auditable reduction evidence.

Technical whitepaper v0.1
August 2026
Apache-2.0 project

Authoritative Markdown SHA-256: 6be2a68cdf097333291fcffd9210df6371d5cc053b8b1cacb416d61a169171df

Contents

The Markdown source is authoritative. Figures are vector renderings generated from explicit source markers.

Abstract

Large language model applications repeatedly transmit system instructions, conversation history, retrieved documents, tool definitions, and verbose tool results. Much of that material is redundant at the byte or representation level, yet an optimizer that removes the wrong token can change an instruction, break a tool call, invalidate structured output, destroy a reusable cache prefix, or trigger an expensive correction turn. Token reduction is therefore not a text-cleanup problem. It is a constrained transformation problem whose useful objective is lower total task cost under an explicit quality and protocol budget.

Kendr Optimizer is an open-source, provider-neutral transformation engine for this problem. It is not an LLM gateway and does not route requests, hold provider credentials, or call a model. It accepts a versioned, typed content envelope and returns an optimized envelope, a machine-readable receipt, verification results, and optional recovery data. Its core method, Verification-Gated Typed Token Reduction, proposes transformations over known content types and applies a candidate only after structure, tool-protocol, protected-artifact, cache, risk, and measured net-gain gates pass. A best-effort global latency guard stops new engine dispatch after the budget is exhausted; it is not a hard cancellation deadline. Eligible typed encodings must expand byte-for-byte to the pre-transform envelope before they can be accepted.

The pre-alpha implementation contains nine independently implemented native engines, exact BPE measurement for `cl100k_base` and `o200k_base`, a Rust library, a CLI, a transform-only loopback service, and audited adapters for seven agent harnesses. A reproducible nine-case peer experiment shows 71.64 percent qualified prompt/context reduction and 61.38 percent qualified command/tool output reduction for the shipped default on that corpus. These figures are payload measurements with preservation proxies. They are not provider-bill savings, universal downstream-quality results, or proof that Kendr is the best optimizer. The paper defines the evidence needed to make stronger claims.

Paper status and claim boundary

This paper describes the repository at version `0.1.0` and benchmark release `v0.1.0-benchmark.5`. The project is pre-alpha. Contracts, engine behavior, and integration pins can change before a stable release.

The phrase verification-gated has a narrow meaning here: executable invariant checks and byte-exact reconstruction for eligible typed transforms. It does not mean mechanized formal verification, a cryptographic proof, or universal semantic equivalence. Declared-scope proxy-qualified reduction is the signed, token-weighted raw reduction exposed when every case a runner declares eligible completes and passes its fixture proxy. Public ranking-qualified reduction adds full-surface coverage: eligible and completed counts must equal every corpus case on that surface, failures must be zero, and every completed case must pass. Qualification is a binary evidence gate; it does not alter the raw percentage and does not establish that every possible downstream model response would be unchanged.

The present evidence supports these statements:

- Kendr can reduce the locally serialized token surface before a model call.
- Some implemented transforms can be independently expanded byte-for-byte.
- The included benchmark can be reproduced from pinned inputs, runners, logs, outputs, environment metadata, and checksums.
- The audited harness adapters fail open and cover the specific seams listed in their compatibility ledger.

The present evidence does not support these statements:

- every locally removed token produces an equal provider-bill saving;
- output already generated by a provider can be made cheaper afterward;
- downstream answer quality is unchanged for every model and task;
- the current small corpus establishes a universal optimizer leaderboard; or
- every future version of a named harness remains compatible.

1. Name and positioning

The project name remains Kendr Optimizer. The executable and concise command name remain **`kendr-opt`**. The technical approach is named Verification-Gated Typed Token Reduction.

This naming was chosen for four reasons:

1. **`Kendr Optimizer`** fits the existing Kendr product family and the current crate, schema, adapter, and repository names.
2. **`typed`** identifies the central difference between transforming an arbitrary string and transforming a message, JSON value, tool call, or tool result.
3. **`verification-gated`** describes the actual acceptance mechanism without claiming a formal proof system.
4. **`token reduction`** names the measured objective while leaving room for byte-preserving no-ops when a transform is unsafe or unprofitable.

An exact-phrase and package-registry collision check performed on 2026-08-08 found no project using the complete technical phrase and no package named **`kendr-optimizer`** on crates.io, npm, or PyPI. This is a practical naming check, not legal or trademark clearance. Package names should be reserved before a public announcement.

The project tagline is:

> *Reduce only what passes structure, integrity, cache, risk, and net-gain gates.*

2. The optimization problem

2.1 What is being optimized

Let `E` be the normalized content envelope visible to Kendr, `S(E)` its stable serialization, and **`tau`** the selected tokenizer. The local preflight token count is:

```
T(E, tau) = | tau(S(E)) |
```

For candidate envelope `E'`, gross local token reduction is:

```
delta_tokens = T(E, tau) - T(E', tau)
reduction_percent = 100 * delta_tokens / T(E, tau)
```

The optimizer may accept `E'` only when the signed delta and percentage clear policy thresholds. A negative value is token inflation and must not be shown as a saving. A zero value is a measured no-op.

This local objective is necessary but insufficient. The practical system-level objective is closer to:

```
minimize expected total task cost

subject to:
  task_quality(optimized) >= task_quality(baseline) - epsilon
  protocol_invariants(optimized) = true
  security_invariants(optimized) = true
  retry_and_correction_cost is included
  cache_effects and optimizer overhead are included
```

No deterministic preflight verifier can compute **`task_quality`** for all models and tasks. Kendr therefore separates what can be established locally from what requires paired downstream experiments.

2.2 Why generic string compression is unsafe

LLM requests combine fields with very different semantics:

- natural-language instructions can contain subtle negation and ordering;
- code and JSON are syntax-sensitive;
- tool call IDs and arguments are protocol state;

- tool results may be verbose but contain a single actionable error;
- schemas define what the model may emit;
- cached prefixes have economic value independent of their raw size; and
- message order and role encode authority and conversation causality.

A transform that is appropriate for prose can be invalid for a tool argument. A command-output filter that looks effective by token percentage can remove a fixture-required literal. A paraphrase that is semantically plausible to one model can alter another model's behavior. Kendr therefore treats content type, role, phase, host capability, and declared policy as part of every decision.

2.3 Input and output cost are different problems

Input optimization changes content before the provider call. A successful preflight reduction may reduce billable input, subject to provider framing, tokenization, caching, and billing rules.

Completed output cannot be retroactively made cheaper. Once the model has generated tokens, post-processing changes only what is displayed or retained for future context. Output cost can be influenced before generation through a host-native verbosity control, a maximum-output setting, or a concise response instruction. Such a policy adds input tokens and can change quality, so Kendr treats it as a separate, opt-in generation recommendation with a break-even gate. When a completed answer is compacted before being replayed as history, the saving belongs to future input, not past output.

3. Scope and non-goals

Kendr Optimizer is deliberately smaller than a gateway.

```
Host, agent, or gateway
        |
        | normalized typed envelope
        v
Kendr Optimizer
  validate -> propose -> measure -> verify -> accept/revert
        |
        | optimized envelope + receipt + optional recovery
        v
Host, agent, or gateway
        |
        | provider-specific request
        v
Any selected LLM
```

The core does not:

- choose a provider or model;
- store or read provider credentials;
- expose a chat-completions endpoint;
- forward traffic to an LLM;
- promise a fixed compression ratio;
- silently call an upstream optimizer;
- require another optimizer at runtime; or
- label a local estimate as a verified provider saving.

The boundary is important for deployment and licensing. A user can put Kendr beside an existing gateway, embed the Rust core, invoke the CLI over standard input/output, or use a loopback transform service. Provider selection and request transport remain under the host's control.

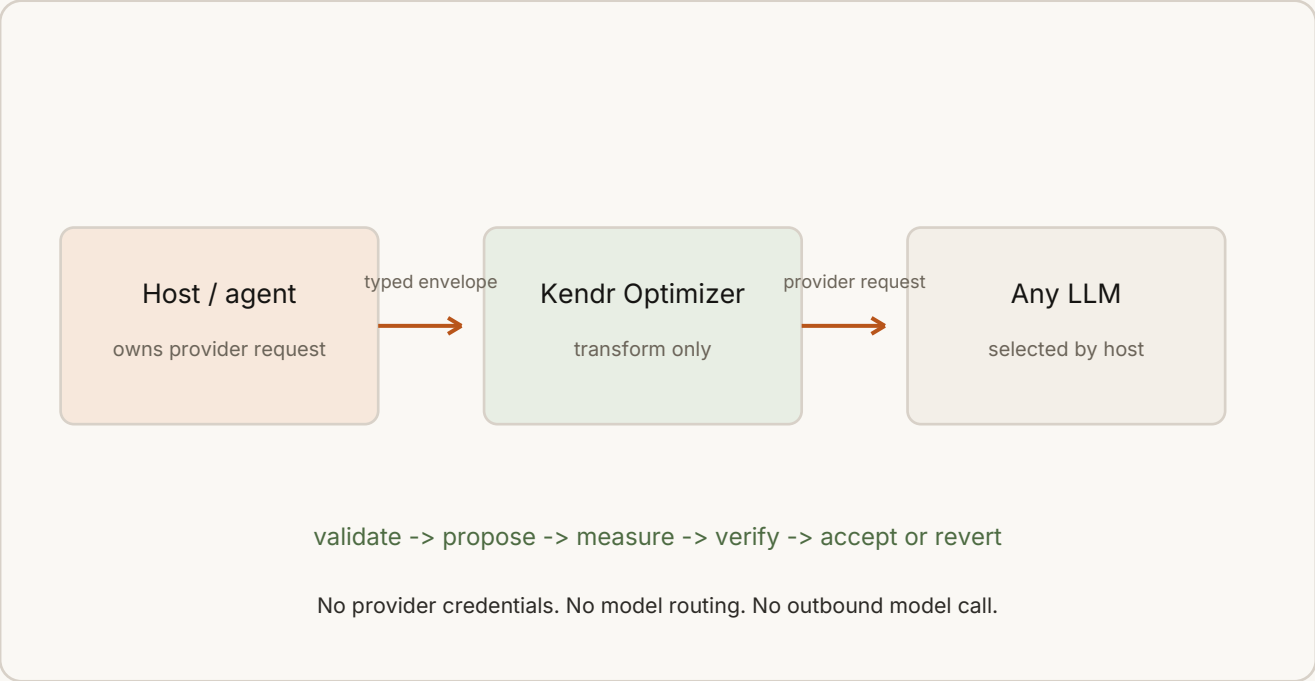

Figure 1. Kendr's transform-only trust boundary.

4. Design principles

4.1 Typed before heuristic

Every adapter maps host data into a neutral envelope with explicit message roles and content-part variants. The engine can then make narrow promises such as "minify this parsed JSON value" or "fold this exact pytest sequence" rather than "shorten this string."

4.2 Candidate before mutation

An engine proposes a full candidate envelope. The controller measures and verifies it before commitment. The original value remains available throughout the transaction.

4.3 Signed whole-envelope accounting

The measured object is the serialized envelope, not a selected substring. This captures marker overhead, schema changes, and transformations that are locally smaller by bytes but larger for the selected tokenizer.

4.4 Failure preserves the request

Unsupported shapes, timeouts, malformed markers, failed invariants, and unprofitable candidates return the prior valid envelope. Adapters also validate the returned host shape and fail open to the original host value.

4.5 Risk is a policy input

Representation-safe and exactly reconstructable transformations are not treated as equivalent to extractive or learned selection. Each engine declares a risk class, and the request policy sets the maximum allowed class.

4.6 Evidence labels are part of correctness

The receipt distinguishes bytes, local tokens, estimated cost, provider usage, paired observed deltas, and quality-adjusted savings. A UI must not collapse these into a single green "saved" label.

4.7 Independent implementation

Peer research and public interfaces may inform design, but production engines are independently implemented and do not wrap Headroom, RTK, Caveman, LLMLingua, OmniRoute, or another optimizer. Comparative runners are isolated under `benchmarks/` and are not runtime dependencies.

5. Versioned normalized contract

5.1 Request envelope

The `kendr.optimize/v1` request contains:

Field	Purpose
`schema_version`	Reject incompatible structural contracts
`phase`	Distinguish request, tool-result, history-ingest, and output-observation work
`request_id`	Correlate receipts without embedding provider credentials
`content`	Messages, typed parts, tools, and output contract
`target`	Tokenizer, optional model/pricing hints, and cache segments
`generation`	Expected output and user verbosity intent
`host_capabilities`	Declare which behaviors the host can safely support
`policy`	Risk, gain, cache, history, and feature gates plus a latency guard

The contract is provider-neutral. Provider-only fields remain outside the envelope in the adapter and are copied unchanged when the optimized fields are reassembled.

5.2 Content types

Messages have stable IDs, roles, optional parent/turn identifiers, typed parts, and metadata. Roles are `system`, `developer`, `user`, `assistant`, and `tool`. Parts are:

- `text`;
- `code` with optional language metadata;
- `json` as a parsed value;
- `document` with text and document metadata;
- `image_reference`;
- `tool_call` with immutable ID, name, and arguments; and
- `tool_result` with call ID, optional name, content, and error state.

Tools have names, descriptions, input schemas, and a required flag. The output contract is carried separately and is immutable under the current verifier.

This representation prevents a prose engine from accidentally rewriting code, JSON, image references, tool arguments, or output constraints.

5.3 Target and cache context

The target may select `cl100k_base`, `o200k_base`, or an approximate profile. It can also carry a model label, pricing hints, and cache segments identified by message IDs. The model label is an accounting hint, not routing authority.

Cache segments matter because a smaller request can cost more if it breaks a high-value reusable prefix. With `preserve_cache_prefix=true`, any candidate touching a protected message or the tool catalog is rejected.

5.4 Host capabilities

Risky behaviors require an explicit host capability. Examples include:

- `can_narrow_tools` for optional tool removal;
- `can_retry_with_full_tools` for recovery after a selection miss;
- `can_restore_references` for history references;
- `can_set_verbosity` for native generation control; and
- `streaming_output` for output-lifecycle decisions.

An adapter must report what the harness can actually do, not what the optimizer would like it to do.

5.5 Default policy

The current default policy uses a `recoverable` risk ceiling, a minimum gain of 8 tokens and 1 percent, a 25 ms optimizer latency budget, cache-prefix protection, a recent-history floor of 6 messages, and a 24,000-character generic tool-result limit. Generic lossy pruning, tool selection, and generation policy are disabled unless the caller opts in.

6. Risk model

The ordered risk classes are:

Risk	Meaning	Default status	Required evidence
Pass-through	Measurement only; content unchanged	Allowed	Stable measurement
Representation-safe	Parsed value or rendering representation is preserved	Allowed	Structural equality or narrow invariant checks
Recoverable	Visible representation changes and omitted bytes have exact recovery	Allowed	Independent expansion or exact recovery plus integrity checks
Extractive	A relevance policy removes information	Disabled	Task-specific quality and recovery plan
Learned	A local model influences selection or rewriting	Planned, disabled	Model card, resource limits, adversarial and downstream evaluation

Risk order is enforced numerically. A candidate whose declared risk exceeds the request ceiling is skipped before it can mutate the active envelope.

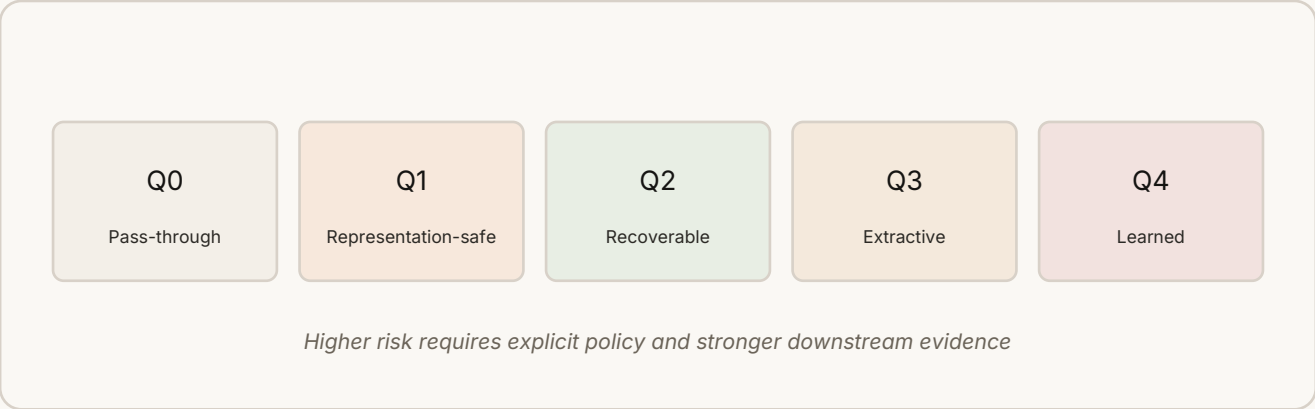

Figure 2. Ordered risk classes; the default ceiling is recoverable.

"Recoverable" does not by itself imply model-visible equivalence. A hidden copy of the original helps operators and retries, but the model cannot use what it did not receive. For the exact repetition encodings, Kendr additionally checks that the model-visible marker is canonical and independently expands to the original bytes. Generic extractive recovery capsules do not receive this stronger claim.

7. Optimization pipeline

7.1 Current controller

The current controller is a deterministic greedy sequence over nine engines. Each accepted candidate becomes the input to the next engine.

```
function optimize(request):
    require request.schema_version == "kendr.optimize/v1"
    validate(request.content)

    original = request.content
    current = original
    baseline = measure(serialize(original), request.target.tokenizer)
    attempts = []

    generation_recommendation = evaluate_generation_policy(request)

    for engine in ordered_engines:
        if not enabled(engine, request.policy, request.host_capabilities):
            continue

        if monotonic_clock() - planning_started >= latency_budget:
            attempts += timed_out(engine, "budget exhausted before dispatch")
            continue

        if engine.risk > request.policy.risk_ceiling:
            attempts += rejected(engine, "risk ceiling")
            continue

        candidate = engine.propose(request, current)

        if candidate is none:
            attempts += no_candidate(engine)
            continue

        before = measure(serialize(current), tokenizer)
        after = measure(serialize(candidate.content), tokenizer)
        checks = verify(engine, current, candidate)

        if protected_cache_touched(request, candidate):
            attempts += rejected(engine, "cache prefix")
        else if any verification check fails:
            attempts += rejected(engine, failed_checks)
        else if not profitable(before, after, policy):
            attempts += rejected(engine, "net gain")
        else:
            current = candidate.content
            attempts += applied(engine, before, after, checks)

    portfolio = measure(serialize(current), tokenizer)
    if not profitable(baseline, portfolio, policy):
        current = original
        mark portfolio as reverted

    return current, receipt, optional_recovery
```

The controller records no-candidate, reject, apply, shadow, revert, and timed-out attempt states. This is why "nothing changed" is diagnosable: the receipt can say whether no engine found a candidate, the risk ceiling blocked it, a cache segment would change, verification failed, the gain was too small, or the pre-dispatch latency guard stopped later engines.

7.2 Acceptance predicate

For candidate `c = (E', risk, touched, reconstruction)`, the current acceptance predicate is conceptually:

```
Accept(c) =
    schema_valid(E')
    and identifiers_valid(E')
    and output_contract(E') == output_contract(E)
    and tool_calls(E') == tool_calls(E)
    and exact_typed_parts(E') == exact_typed_parts(E)
    and tool_surface_allowed(E', E, engine)
    and protected_artifacts(E') covers protected_artifacts(E)
```

```
    and typed_expansion_is_exact(c), when required
    and risk(c) <= policy.risk_ceiling
    and cache_policy_allows(c.touched)
    and delta_tokens >= policy.min_gain_tokens
    and reduction_percent >= policy.min_gain_percent
```

The verifier is intentionally asymmetric: a candidate must satisfy every applicable condition. It is not enough for most checks to pass. Latency is a planner guard outside this predicate: the current synchronous engine API checks the shared budget before dispatch but cannot cancel work already in progress.

7.3 Portfolio-level rollback

Per-engine gain does not guarantee final gain when serialization interactions accumulate. Kendr remeasures the complete final envelope against the original. If the portfolio does not clear the same signed thresholds, it restores the original and marks the portfolio reverted.

7.4 Planner direction

Greedy ordering is simple and deterministic but not globally optimal. Two engines can target overlapping bytes, invalidate one another's preconditions, or produce a locally profitable transform that prevents a better later one. The planned planner will represent candidates as a conflict graph:

```
candidate node:
  target ranges
  risk
  predicted token delta by tokenizer
  latency/resource cost
  cache impact
  verification contract

edge:
  overlapping target, invalidated precondition, or incompatible recovery
```

Selection can then maximize verified net benefit under policy constraints. Any learned ranker will rank already-bounded candidates; it will not bypass the deterministic verifier.

8. Verification and integrity

8.1 Structural checks

The envelope validator rejects empty or duplicate message IDs and empty or duplicate tool names. Candidate verification then enforces an unchanged output contract, immutable tool calls, and exact fingerprints for code, JSON, and image-reference parts.

For ordinary engines, the complete tool list must be unchanged. The tool selector is the only exception: it may remove optional tools, but every retained definition must equal its original and every required tool must remain.

8.2 Protected-artifact multiset

For text, document, and tool-result content, Kendr extracts a multiset of high-risk literals:

- HTTP and HTTPS URLs;
- Windows and Unix paths;
- signed numbers, decimals, scientific notation, units, and percentages;
- long hexadecimal or uppercase identifiers;
- explicit negations such as `not`, `never`, and `cannot`;
- `KENDR_PRESERVE_BEGIN` through `KENDR_PRESERVE_END` blocks; and
- complete lines containing error, fatal, panic, exception, or failed signals.

Multiplicity matters. If a URL appeared twice before a generic transformation, one remaining copy does not satisfy the verifier. ANSI escape sequences are removed before artifact collection so a safe terminal-color

cleanup does not look like artifact loss.

Regex extraction is defense in depth, not a complete semantic parser. It can miss identifiers, localized negation, or domain-specific constraints. This is one reason lossy engines remain opt-in and why downstream evaluation is still required.

8.3 Typed transform proof

For exact repetition, repeated-line, and pytest folding engines, the verifier does not trust a marker merely because it looks plausible. It:

1. examines only content parts that changed in this candidate;
2. dispatches to the engine-specific parser;
3. requires the exact canonical marker grammar;
4. applies resource and forward-reference bounds;
5. expands every marker independently; and
6. compares the reconstructed full envelope byte-for-byte with the input to that engine.

Authored marker-shaped text in an unchanged part is never interpreted as a proof. Malformed counts, leading-zero variants, digest mismatch, mixed line endings, invalid ranges, and expansion beyond limits fail closed.

8.4 Recovery capsules

When an accepted portfolio has recovery data, the current pre-alpha capsule can contain the complete original serialized envelope and a SHA-256 digest. Restore parses the payload, recomputes the digest, and rejects any mismatch.

This design is easy to verify but has an important privacy cost: the capsule can contain the entire sensitive prompt. Operators must not log it, share it across tenants, or retain it without a TTL. Planned recovery uses scoped content- addressed fragments, authenticated encryption supplied by the host, quotas, and explicit deletion semantics.

The embedded SHA-256 digest detects accidental corruption only when the digest itself has not been replaced. Because the digest travels with the plaintext capsule, it provides no authenticity or tamper resistance against an attacker who can change both fields. A host-held HMAC key, signature, or authenticated encryption layer is required before recovery data can cross a trust boundary.

9. Native engine portfolio

The current order is fixed and visible:

Order	Engine	Surface	Risk	Core guarantee
1	`json-minify`	Embedded JSON text	Representation-safe	Parsed JSON value is preserved
2	`terminal-clean`	Tool results	Representation-safe	ANSI presentation bytes are removed
3	`text-normalize`	Text/documents	Representation-safe	Conservative redundant blank space only
4	`repeat-lines`	Tool results	Recoverable	Exact repeated lines expand byte-for-byte
5	`pytest-result-fold`	Pytest output	Recoverable	Validated sequential results expand exactly
6	`context-repetition`	Text/documents	Recoverable	Exact repeated units expand byte-for-byte
7	`history-dedup`	Conversation history	Recoverable	Host-restorable replay references only
8	`tool-output-prune`	Large tool results	Extractive	Diagnostic retention, no semantic-equivalence claim
9	`tool-selector`	Optional tools	Extractive	Required tools retained; retry capability required

9.1 `json-minify`

This engine finds eligible embedded JSON, parses it with `serde_json`, and serializes the same value without insignificant formatting whitespace. Parsed JSON parts are already immutable under the verifier; this engine is for JSON embedded in transformable text surfaces. It rejects invalid input and accepts only a token-profitable candidate.

The guarantee is value preservation under the parser and serializer, not byte identity. Object member order, number spelling, or escape representation can change even when the JSON value does not.

9.2 `terminal-clean`

Terminal output often contains ANSI color and cursor-control sequences that add tokens but do not carry the plain-text diagnostic. The engine strips recognized ANSI sequences from tool-result content. Protected artifacts are collected from ANSI-sanitized text on both sides, preventing presentation codes from hiding a literal mismatch.

This is not a general terminal emulator. Unknown or semantic control behavior is outside the current contract.

9.3 `text-normalize`

The normalizer performs conservative whitespace cleanup over eligible prose without touching typed code or JSON. It targets redundant blank-line patterns, not arbitrary whitespace. The whole-envelope token gate rejects changes that do not improve the selected tokenizer count.

9.4 `repeat-lines`

For a run of at least four identical, non-empty tool-result lines, the engine retains the first line and emits a count marker containing the number omitted and a SHA-256 digest of the retained source line. Its independent expander requires canonical decimal counts and lowercase hexadecimal digests, validates the source digest, and recreates the exact run. Checked arithmetic rejects any expansion above 1,000,000 lines or 64 MiB before repeated copies are allocated.

Conceptual marker form:

```
<retained line>
<kendr repeat marker: omitted=N, sha256=SOURCE_DIGEST>
```

The actual marker grammar is versioned in the engine implementation. Candidate recovery also carries the prior envelope, but verifier acceptance depends on the typed marker expansion, not only on that hidden copy.

9.5 `pytest-result-fold`

Pytest output requires more care than keeping a summary. The engine first parses result lines and the final summary. It refuses malformed, incomplete, oversized, or internally inconsistent output. Only `PASSED`, `SKIPPED`, and `XFAIL` sequences are foldable. `FAILED`, `ERROR`, and `XPASS` result lines stay visible.

An eligible sequence must contain at least eight result lines with matching status, prefix, suffix, numeric width, and consecutive numeric values. Two edge lines are retained on each side. The marker records mode, status, omitted count, numeric range, width, hex-encoded prefix and suffix, and a digest of the omitted block. The expander regenerates every line, validates the digest and summary constraints, and enforces byte and line limits.

This engine addresses a concrete weakness in generic test-output pruning: high-volume success evidence can be folded aggressively while failure evidence and exact reconstructability remain explicit.

9.6 `context-repetition`

Context repetition targets exact redundancy in text and document parts. It does not paraphrase. The engine evaluates four representations and keeps the best profitable exact candidate:

1. a repeated prefix block represented by one block marker;
2. repeated paragraphs referencing an earlier paragraph ordinal;
3. adjacent repeated lines referencing the retained line; and

4 runs of at least three exact sentences represented by source byte length and additional-copy count.

The input must be at least 128 bytes and no more than 2 MiB. The candidate must save at least 16 bytes before the controller's token gate. Unit counts are bounded at 8,192. Sentence recognition is deliberately narrow: ASCII `.`, `!`, or `?` followed by horizontal whitespace or line end, at least 32 non-padding bytes, and no inspection of fenced backtick or tilde lines.

For prefix blocks, the implementation uses a linear-time Z-algorithm to find a repeated prefix period without quadratic substring comparison. Expansion uses source bytes immediately preceding the marker, checks UTF-8 boundaries and declared paragraph counts, and rejects additional markers in the exclusive block form. Paragraph and line markers use backward ordinals only; forward references are invalid.

Canonical ASCII forms include:

```
[[kendr.repeat unit=paragraph source=<ordinal> copies=<count>]]
[[kendr.repeat unit=line source=<ordinal> copies=<count>]]
[[kendr.repeat unit=block paragraphs=<count> bytes=<bytes> additional_copies=<count>]]
[[kendr.repeat unit=sentence bytes=<bytes> additional_copies=<count>]]
```

9.7 `history-dedup`

Conversation history can replay content already retained by the host. This engine may replace eligible history with references only when the host declares `can_restore_references=true`. Recent-message floors, roles, and policy bounds constrain selection. It is recoverable at the application layer, not a promise that an arbitrary provider understands a Kendr reference.

Adapters that cannot restore references report the capability as false, so the engine does not run.

9.8 `tool-output-prune`

This generic reducer is extractive and disabled by default. It is intended for oversized tool results that lack a safer typed reducer. It retains protected diagnostics and policy-selected context, but removal can affect model behavior even when a recovery capsule exists. It should be used only with task-specific quality evaluation and a host recovery path.

9.9 `tool-selector`

Large tool catalogs consume input on every call. This engine can remove only optional tools; required tools and every retained definition are immutable. It is disabled unless the policy opts in and the host can both narrow the tool set and retry with the complete set. A selection miss can otherwise cause an expensive or unrecoverable task failure.

The long-term selector should optimize expected total cost, not schema bytes:

```
expected benefit = schema tokens avoided
                 − miss_probability * expected retry cost
                 − cache invalidation cost
                 − selector overhead
```

10. Token measurement, receipts, and cost truth

10.1 Stable measurement object

Kendr serializes the complete `ContentEnvelope` with `serde_json` and records:

- byte length;
- SHA-256 digest;
- token count;
- tokenizer identifier; and
- measurement confidence.

Exact local measurement is available for `cl100k_base` and `o200k_base` through `tiktoken-rs`. Without a known profile, the approximate counter takes the maximum of a character-derived estimate and a whitespace floor. Approximate counts are labeled approximate and must not be presented as exact provider usage.

Provider framing can add tokens outside the normalized envelope. Providers may also tokenize differently, apply hidden prefixes, cache prefixes, round usage, or bill reasoning and output classes separately. The receipt therefore keeps local counts separate from provider observations.

10.2 Receipt states

The `kendr.receipt/v1` receipt contains baseline and final measurements, signed deltas, percentage, status, warnings, cache impact, estimated money, per-engine attempts, verification checks, and recovery metadata. Attempt states are `applied`, `no_candidate`, `rejected`, `shadow`, `reverted`, and `timed_out`.

The top-level result is one of:

- `applied`: at least one candidate survived and the portfolio remained profitable;
- `shadow`: analysis was requested without returning changed content;
- `skipped`: no input candidate survived the gates (including an ordinary no-op); or
- `reverted`: local candidates existed but the accepted portfolio failed a final gate.

10.3 Evidence ladder

Kendr uses an evidence ladder rather than one savings flag:

Level	Name	What is known
E0	Byte delta	Serialized bytes changed
E1	Local token delta	A declared tokenizer recount changed
E2	Estimated cost delta	E1 mapped through declared pricing assumptions
E3	Observed optimized usage	Provider usage for the optimized run only
E4	Paired observed delta	Baseline and optimized provider usage are paired
E5	Quality-adjusted saving	Paired saving passes task success/non-inferiority criteria

Preflight optimization reaches E1, or E2 when pricing hints exist. It cannot reach E4 without a paired baseline. The current `observe` path marks its `verified` flag only when the pair shows a positive provider-reported cost reduction (when comparable cost is present), or otherwise a positive combined input-plus-output token reduction, and the optimized task reports success. A paired zero or increase remains an E4 delta but is not a verified saving. This single-pair binary gate is not statistical non-inferiority; publication-grade E5 evidence requires repeated, randomized, task-native trials described later.

Figure 3. Local token reduction and provider-verified savings are different evidence levels.

10.4 Correct user-facing language

An applied transform should be reported as, for example:

```
13 input tokens reduced (preflight, o200k_base).
Provider saving not yet verified.
```

"No measured saving" is correct only when the local delta is zero or negative. It is not a correct substitute for "provider saving not yet verified." This distinction directly addresses the misleading behavior that motivated the project review.

11. Harness and gateway compatibility

Compatibility is a property of an integration seam and version, not a blanket brand claim. All adapters send only normalized content to a numeric loopback optimizer endpoint, keep provider credentials outside Kendr, validate returned shape, and fail open.

Harness	Audited version	Applied surfaces	Important boundary
Claude Code	2.1.224	Successful `PostToolUse.updatedToolOutput`	Prompt, full history, tool catalog, failed tool output, and generation settings are not replaceable through stable hooks
Claude Code Channels	Research preview in 2.1.224	Authorized notification content before channel emission	Cannot intercept another channel server; authentication remains upstream
Pi coding agent	0.84.1	System prompt, context messages, text tool results	No tool narrowing or generic provider-payload rewrite
OpenCode	1.18.15 V1	Current user text and tool output; opt-in experimental history/system	Tool schemas and generation rewrites disabled
Hermes Agent	package 0.20.0, tag v2026.8.3	Main-agent request text, recognized tools, string tool results	Auxiliary model calls and opaque multimodal parts are outside scope
OpenClaw	2026.7.2	Assembled history and supported tool-result text	Context-engine slot is exclusive; initial prompt and arbitrary first-call schemas are outside ordinary seam
NanoClaw	pinned unreleased main	Initial and normal follow-up formatted prompt strings	Source customization is required; opaque resumed history and tools remain outside scope

The compatibility ledger stores immutable upstream commits. The latest contract-level verification records 59 passing adapter tests and zero failures. That is not a universal end-to-end certification; upgrades must advance the pin and rerun the relevant suite.

An external gateway can integrate through one of three stable surfaces:

1. embed `kendr-optimizer-core` and the contracts crate;
2. invoke `kendr-opt analyze|optimize|restore|observe` with JSON; or
3. call the loopback-only transform API.

The local service exposes health, capabilities, engines, analyze, optimize, restore, and observe routes. There is intentionally no provider-forwarding route. The current service has no built-in authentication, body-size limit, or concurrency quota, so it must remain on loopback or behind an operator-managed security boundary.

12. Competitive landscape and design gaps

Token optimization describes several different products. Comparing them only by one percentage is misleading.

System	Primary layer	Typical method	Comparison boundary
LLMLingua	Prompt/context	Small causal LM scores and removes tokens under a budget	Learned prompt compression; model and target rate matter
LongLLMLingua	Long RAG context	Query-aware ranking, reordering, and fine-grained compression	Eligible mainly for retrieved-document scenarios
LLMLingua-2	Prompt/context	Bidirectional token classifier trained by data distillation	Learned extractive compression; model load and domain matter
Headroom	Broad context layer	Structural routers plus optional learned Kompress model and recovery	Closest broad peer; library/proxy/MCP surfaces exceed Kendr's transform-only boundary
RTK	Command output	Command-aware filters, grouping, truncation, and deduplication	Tool-output specialist, not a general prompt compressor
Caveman	Future generation style	Persistent terse instructions and language rules	Changes what a model generates; not post-hoc payload compression
OmniRoute	Gateway plus compression	RTK-style and Caveman-style modules in a routing product	Benchmark only the invoked pure compression modules, not gateway value
Selective Context	Prompt/context research	Self-information based token selection	Older Python stack and learned-lossy assumptions
RECOMP	RAG context research	Extractive or abstractive compression before retrieval augmentation	GPU/model constraints and RAG-specific evaluation
PCToolkit	Research toolkit	Wraps multiple prompt-compression methods	Meta-harness; ranking it would double-count algorithms

Kendr's intended contribution is not a new universal salience model. It is the combination of a provider-neutral typed contract, independently implemented deterministic transforms, per-candidate executable gates, signed whole-envelope accounting, explicit recovery, cache-aware refusal, honest receipts, and harness-specific capability declarations.

Peer projects contain important ideas and often broader features. Kendr's claim must remain narrower until larger paired experiments show an advantage in total quality-adjusted cost.

13. Benchmark and ranking methodology

13.1 Reproducible release

The current benchmark release is `releases/v0.1.0-benchmark.5`. It contains:

- the authored nine-case corpus;
- 13 complete raw arm files;
- 17 successful executable attempts out of 17;
- exact peer and harness locks;
- environment and execution metadata;
- command stdout and stderr;
- derived JSON, CSV, and Markdown reports;
- a minimal source and runner snapshot for reproduction; and
- 139 SHA-256 entries, all independently rechecked after assembly.

The snapshot is the exact executed source, not a post-run formatting export. Its build and checksums are release evidence; later source-tree formatting can therefore differ without changing the scientific result.

The benchmark recounts visible input and output with `tiktoken o200k_base 0.12.0`. It does not call a target LLM or observe a provider bill. Five cases are eligible for the prompt/context surface and four for the command/tool-output surface.

13.2 Measurement and qualification pipeline

The test has three distinct outputs: a raw token reduction, one composite fixture-preservation result per completed case, and an optional qualified token reduction. Keeping them separate prevents a high compression number from being mistaken for evidence of preservation.

Raw reduction

For the set `C` of completed cases with a score, the independent ranker recounts the complete visible benchmark strings with `tiktoken o200k_base 0.12.0`:

```
I = sum(o200k_base_tokens(input_text[c]))  for c in C
O = sum(o200k_base_tokens(output_text[c])) for c in C

raw_reduction_percent = 100 * (I - O) / I
```

The result is signed and token-weighted; it is not an average of per-case percentages. A negative result means token inflation. The complete measured string includes the benchmark's exact `Question: ...` suffix.

Completed zero-token-delta cases remain in `I` and `O`. Failed or otherwise uncompleted cases have no scored output and therefore do not enter the token-sum denominator. Instead, they remain in coverage and failure accounting and prevent qualification. Unsupported cases are excluded from a runner's declared-eligible aggregate, while the public ranker separately requires every corpus case on the surface.

Composite fixture-preservation gate

A completed case passes its composite gate only when all applicable conditions are true:

1. every unique fixture-declared required literal still appears in the primary optimized output;
2. when the fixture requires JSON equivalence, both values parse and the optimized JSON value equals the original JSON value; and
3. the exact benchmark query marker remains in the complete measured output.

The scorer also reports URL, path, and number recall as diagnostics. Those are not separate hard gates unless the fixture also declares the exact values as required literals. Thus `5/5` means five completed cases passed their

composite case-level gate, not five individual invariant checks. This shared peer proxy is distinct from Kendr's internal verifier and is not a downstream LLM-quality assessment.

From raw to qualified

The release summary first records declared-scope qualification:

```
declared_scope_qualified =
    completed_cases == declared_eligible_cases
    and every completed case passes its composite fixture gate
```

The public primary ranking applies the stricter full-surface gate, where `N` is five prompt/context cases or four command/tool-output cases:

```
full_surface_coverage =
    eligible_cases == N
    and completed_cases == N
    and failed_cases == 0

public_rank_qualified =
    full_surface_coverage
    and proxy_passed_cases == completed_cases

qualified_reduction_percent =
    raw_reduction_percent, if public_rank_qualified
    N/A,                   otherwise
```

Qualified reduction is therefore not a recomputed or discounted percentage. It is the unchanged raw percentage admitted only after the binary gate passes. When it is `N/A`, the report keeps the raw percentage as a diagnostic and names the failed coverage or proxy count, but the row receives no primary rank. LongLLMLingua illustrates the two levels: its one supported RAG case passed, so the release summary can report `1/1` declared-scope qualification, while the public prompt/context table excludes it for `1/5` full-surface coverage.

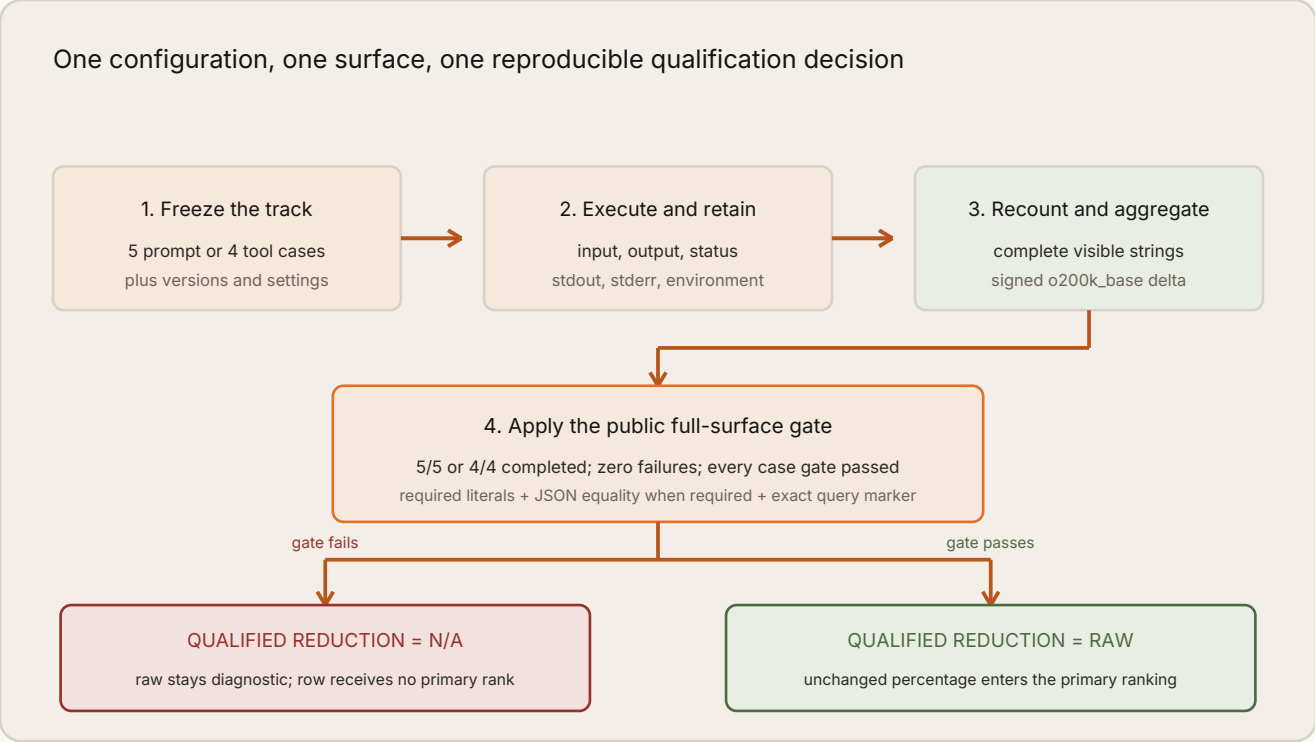

Figure 4. Qualification is a binary admission gate: it preserves or withholds the raw percentage; it never discounts it.

13.3 Ranking rules

The public ranking implementation follows these rules:

1. Prompt/context and command/tool-output are separate leaderboards.
2. Only the predefined comparable case set can receive a primary rank.
3. The unoptimized pass-through row is a baseline, not a competitor.

4 Kendr is represented only by its shipped default; peer configurations remain separately visible and carry target-rate, structural-only, or feasibility labels where applicable.

5 Coverage must include every frozen case on the surface, with zero failures.

6 Every completed case must pass its composite fixture gate for a qualified rank.

7 Excluded raw reduction remains visible in a separately labeled, non-ranked diagnostic table. Display order does not confer an ordinal rank or imply quality.

8 Generation-policy experiments such as Caveman remain a separate track.

9 Latency is diagnostic when process startup and model warm-up are not normalized and never changes rank.

10 Rows sort by the stored four-decimal qualified reduction and then by fewer completed cases with `token_delta == 0`. Rows equal on both values share a standard competition rank; optimizer ID and setting affect display order only. Latency and no hidden composite score affect rank.

The result is intentionally not one universal number. It is a stratified, auditable view of reduction, preservation, coverage, risk, and evidence level.

13.4 Prompt/context results

Primary comparable rows from benchmark release `.5`:

Qualified rank	Optimizer / setting	Input	Output	Raw reduction	Qualified reduction	Cases passing composite fixture gate
1	Kendr Optimizer `default`	6,358	1,803	71.64%	71.64%	5/5
2	LLMLingua GPT-2 feasibility `target-50`	6,358	2,261	64.44%	64.44%	5/5
3	Headroom structural-only `target-50`	6,358	3,906	38.57%	38.57%	5/5
4	OmniRoute deterministic `rtk-standard+caveman-full`	6,358	6,260	1.54%	1.54%	5/5
5	Headroom structural-only `default`	6,358	6,358	0.00%	0.00%	5/5

Figure 5. Every shown row completed all five cases and passed all five composite fixture gates.

Diagnostic raw rows excluded from the primary ranking:

- Headroom Kompress plus structural reduced 60.03 percent raw but passed 3/5 completed-case gates; qualified reduction is `N/A` and the row is excluded.

- LLMLingua-2 small reduced 43.10 percent raw but passed 0/5 completed-case gates; qualified reduction is `N/A` and the row is excluded.
- LongLLMLingua GPT-2 reduced 48.89 percent on its single eligible RAG case. It is reported but excluded from the primary table for 1/5 full-surface coverage.
- Headroom structural default was a qualified zero-reduction row; pass-through was the zero-reduction baseline.

The LLMLingua and LongLLMLingua GPT-2 configurations are explicitly feasibility substitutions for larger canonical models. Configured `target-50` arms represent a requested operating point, not an optimizer discovering the ideal ratio.

13.5 Command/tool-output results

Qualified rank	Optimizer / setting	Input	Output	Raw reduction	Qualified reduction	Cases passing composite fixture gate
1 of 1	Kendr Optimizer `default`	12,569	4,854	61.38%	61.38%	4/4

The phrase "1 of 1" is deliberate. Other executed optimizer rows completed the track but failed at least one composite fixture gate:

Optimizer / setting	Raw reduction over completed cases	Cases passing composite fixture gate	Why excluded from primary ranking
RTK documented filter per fixture	97.27%	0/4	Preservation gate failed on 4/4 cases; qualified reduction `N/A`
OmniRoute deterministic stack	71.45%	1/4	Preservation gate failed on 3/4 cases; qualified reduction `N/A`
Headroom structural default	17.26%	3/4	Preservation gate failed on 1/4 cases; qualified reduction `N/A`
Headroom structural target-50	17.26%	3/4	Preservation gate failed on 1/4 cases; qualified reduction `N/A`
Headroom Kompress plus structural	21.58%	2/4	Preservation gate failed on 2/4 cases; qualified reduction `N/A`

Figure 6. A row below 4/4 composite fixture-gate passes receives qualified N/A and no primary rank.

These results do not say that those projects are generally unsafe. They say the exercised settings did not preserve every requirement of this authored corpus. The raw outputs are retained so the failure is inspectable.

13.6 Kendr per-case results

Case	Surface	Default reduction
Short request	Prompt/context	0.00%
Redundant prose	Prompt/context	94.67%
Retrieved documents	Prompt/context	59.03%
Pretty JSON	Tool output	40.95%
Terminal output	Tool output	94.38%
Pytest output	Tool output	89.03%
Git log	Tool output	0.00%
Code context	Prompt/context	50.43%
Multilingual repeated context	Prompt/context	93.87%

Two no-ops are desirable signals, not automatic failures: the short request and git-log fixture did not offer a profitable default-safe transform. The shape of these results also identifies the next engineering gap: command-aware git, build, diff, and table reducers can improve coverage without making generic extractive pruning the default.

13.7 Caveman generation-policy track

The pinned Caveman snapshot contains 10 upstream model-output pairs. Recounting with `o200k_base` found 2,045 terse-control output tokens and 1,026 Caveman output tokens, a 49.83 percent additional output reduction. The skill added 14,600 input tokens across the single-turn runs, producing a simple input-plus-output net of negative 13,581 tokens versus the terse control.

This is not a fresh model run, quality was not scored, and a persistent skill can amortize input overhead across turns differently. It is therefore a separate generation-policy observation, not a payload-compression rank.

14. Interpreting the current result

The current evidence changes the improvement strategy in three ways.

First, the earlier "no measured saving" experience was partly a reporting problem. Preflight reduction and provider verification were conflated. The receipt and UI must report both independently.

Second, the largest reliable gains on this corpus came from typed exact redundancy, not generic prose cleanup. Adding exact context repetition and pytest folding changed Kendr from frequent no-op behavior to 71.64 and 61.38 percent qualified reduction on the two small tracks.

Third, raw compression is an adversarial objective when separated from preservation. RTK and OmniRoute produced the highest raw command-output reductions but did not retain the declared requirements for these fixtures. That does not make low reduction preferable by itself; it shows why a public leaderboard must display qualification beside percentage.

The appropriate next strategy is a portfolio of narrow typed reducers, each with a parser, explicit invariants, and adversarial fixtures. A generic learned compressor can later rank or select bounded candidates, but it should not become the only safety boundary.

15. Security and threat model

15.1 Assets

Sensitive assets include prompt and history content, tool schemas, tool arguments and results, output contracts, cache topology, recovery material, and receipts that may reveal workload size or cost.

15.2 Adversaries and failure sources

The system must handle malicious prompt content, marker spoofing, oversized inputs, pathological repetition, regex worst cases, schema drift, compromised extensions, stale harness assumptions, malicious recovery data, and ordinary implementation mistakes.

15.3 Current controls

- the Rust core has no network dependency;
- content is never executed;
- typed parts and tool protocol fields have immutable checks;
- marker grammars are canonical and bounded;
- expansion compares the full prior envelope;
- protected artifacts use multiplicity;
- cache segments can be made immutable;
- risk and feature gates default conservatively;
- a best-effort shared latency guard stops dispatching additional engines after its budget is exhausted;
- a rejected candidate never becomes model-visible, and a portfolio-level failure returns the original content; and
- every dispatched engine has a receipt-visible disposition.

15.4 Known gaps

- engine latency is measured but there is no hard per-engine cancellation;
- the HTTP service has no built-in authentication, body limit, or concurrency limit;
- message topology validation is incomplete;
- protected-artifact regexes are not a semantic parser;
- full-envelope plaintext recovery increases sensitive-data exposure;
- external extension sandboxing does not yet exist;
- learned engines and weights do not yet have a production security model; and
- no local verifier can establish universal semantic equivalence.

Until these gaps close, operators should keep the service on loopback, avoid persisting recovery capsules, use the default risk ceiling, and treat receipts as sensitive telemetry.

16. Performance and resource model

The deterministic engine pack is designed for linear or near-linear passes over its input. Exact tokenization dominates some small transforms. The context prefix detector uses a Z-algorithm to avoid quadratic repeated-prefix search. Marker expansion uses checked arithmetic and explicit byte, line, and unit limits.

Current latency receipts diagnose time spent, and the pre-dispatch guard avoids starting later engines after the shared budget is already exhausted. It does not enforce a hard request deadline because an in-flight synchronous engine cannot be cancelled. The peer benchmark does not rank latency. CLI process startup is included for some arms, while learned-model initialization is warmed outside the per-case timer for others. A fair latency release needs:

- common in-process or persistent-process boundaries;
- cold and warm measurements;
- peak resident memory and model weight size;
- tokenizer initialization accounting;
- distribution statistics rather than only a median; and
- identical hardware and thread limits.

17. Downstream evaluation protocol

Payload proxies are a development gate. A product-quality claim needs paired target-model evaluation.

17.1 Trial unit

For each workload instance, create optimizer-off and optimizer-on variants from the same source. Randomize order within blocks of task, model, and cache state. Use deterministic settings where supported and repeat stochastic tasks with multiple seeds.

17.2 Required observations

Record:

- provider-reported input, cache-write, cache-read, reasoning, and output tokens;
- monetary cost under the actual bill or pinned price schedule;
- first-token and end-to-end latency;
- retries, tool errors, correction turns, and recovery requests;
- task-native success and exact constraint/fact retention;

- optimizer latency, memory, engine sequence, and receipt; and
- model, provider, region, version, harness pin, and cache condition.

17.3 Quality analysis

Deterministic tasks should use exact tests, schema validity, execution success, and required artifact recall. Retrieval tasks should score answer accuracy and citation support. Open-ended tasks require blinded pairwise or rubric-based evaluation with inter-rater agreement. Agent workflows should measure final task success, commands, turns, and total cost, not just the first prompt.

17.4 Non-inferiority

Define a task-specific quality margin `epsilon` before looking at results. Estimate paired quality difference and its confidence interval. A saving is quality-adjusted only if the lower confidence bound remains above the declared non-inferiority boundary. Report worst-case and subgroup regressions even when the aggregate passes.

17.5 Cost endpoint

The primary endpoint should be total successful-task cost:

```
total_cost = input_cost
           + cache_write_cost
           + cache_read_cost
           + reasoning_cost
           + output_cost
           + retries_and_corrections
           + local_optimizer_resource_cost
```

This endpoint can reveal cases where an impressive first-call reduction loses money by causing one additional turn.

18. Open-source architecture and repository

The public repository is intentionally ordinary and assistant-neutral:

```
crates/
  kendr-optimizer-contracts/   versioned Rust contract types
  kendr-optimizer-core/        network-free engines and controller
  kendr-optimizer-cli/         CLI and local transform service
spec/                          language-neutral JSON schemas
integrations/                  thin, version-pinned harness adapters
benchmarks/                    corpus, peer locks, runners, ranking builder
releases/                      immutable evidence bundles
examples/                      request and observation examples
docs/                          architecture, algorithms, security, whitepaper
scripts/                       reproducible documentation and hygiene tools
output/pdf/                    publication PDF
```

The repository does not include development-assistant control directories or instruction files. Product adapters may include files that a target harness requires at installation time, such as NanoClaw's distributable `SKILL.md`; those are runtime integration artifacts, not instructions for developing this repository. A repository hygiene check enforces this distinction and rejects caches or compiled Python bytecode.

Production and benchmark dependency boundaries are separate. A peer package or model may be downloaded into an isolated benchmark environment but must never become a runtime dependency of the core.

19. Extension model

A future extension SDK should require more than a function from string to string. An engine manifest should declare:

- stable ID and version;
- accepted phases, roles, and content types;
- risk class and whether it is deterministic;
- preconditions and touched ranges;
- reconstruction and verifier entry points;
- cache behavior;
- time, memory, and expansion limits;
- fixture and fuzz corpus identifiers;
- license and provenance; and
- whether external code or weights execute.

External engines should run out of process or in a constrained WASM boundary. The core must independently remeasure and verify returned candidates. Extension metadata is not trusted merely because the extension declares it.

20. Roadmap

Near term

- add typed reducers for git, build systems, diffs, tables, and additional test runners;
- implement hard cancellation, service authentication hooks, request-size and concurrency limits;
- replace full-envelope recovery with scoped encrypted fragments;
- strengthen topology and role-sensitive validation;
- add fuzzing and property-based round-trip tests;
- publish Node, Python, and WASM bindings around the same contracts;
- add provider serializer accounting and cache-aware price models; and
- run harness end-to-end fixtures at every compatibility-pin update.

Evidence milestone

- expand the corpus with public and independently sourced workloads;
- preregister paired quality margins;
- run multiple target models and providers;
- publish optimizer-on/off task traces and provider usage;
- add task-native and human evaluation; and
- promote claims only when quality-adjusted total cost improves with statistical confidence.

Longer term

- replace greedy sequencing with a conflict-aware portfolio planner;
- learn candidate ranking from shadow receipts without relaxing deterministic gates;

- add query-aware document selection with explicit extractive risk;
- support signed engine manifests and isolated extensions; and
- stabilize the v1 contract only after independent adapter implementations and recovery migrations have been exercised.

21. Reproduction

Build and test the core with Rust 1.88 or newer:

```
cargo fmt --all -- --check
cargo clippy --workspace --all-targets -- -D warnings
cargo test --workspace --locked
```

Inspect the optimizer:

```
cargo run -p kendr-optimizer-cli -- engines
cargo run -p kendr-optimizer-cli -- analyze --input examples/request.json
cargo run -p kendr-optimizer-cli -- optimize --input examples/request.json
```

Run the local transform service:

```
cargo run -p kendr-optimizer-cli -- serve --bind 127.0.0.1:7331
```

Build a peer release according to the runner help and pinned peer ledger:

```
python benchmarks/runners/execute_release.py --help
```

Regenerate the rankings from an immutable release summary using the documented ranking builder, then verify that every derived artifact records the source summary digest. Regenerate this PDF from `docs/whitepaper.md` with the included documentation build script. Generated PDF inspection images belong under `tmp/pdfs/` and are not release artifacts.

22. Limitations

The current corpus is small and authored by the project. Preservation proxies catch obvious literal and structure loss but cannot substitute for downstream models. Some peer configurations are feasibility substitutions. The current default is strong on exact repetition and tested output patterns but does nothing on some concise or unsupported content. Recovery privacy is incomplete. Service hardening is incomplete. The Rust contracts are pre-alpha. The project has not yet demonstrated provider-verified savings or non-inferior task quality at scale.

These limitations are not footnotes to hide. They define the next work. The optimizer is useful when its narrow receipts are interpreted correctly; its credibility depends on refusing broader claims until the evidence exists.

23. Conclusion

Token reduction should be treated as a constrained systems transformation, not as a race to delete the most text. Kendr Optimizer makes the content boundary, risk class, candidate, verifier, cache impact, signed delta, and evidence level explicit. Its strongest current mechanism is exact typed redundancy encoding: the optimizer can remove repeated model-visible bytes only when a bounded, canonical representation independently reconstructs the original envelope.

The initial benchmark is promising but intentionally modest in what it proves. On one reproducible authored corpus, the shipped default leads the qualified prompt/context rows and is the only qualified command/tool-output optimizer row among the exercised comparable settings. The next milestone is not a louder percentage. It is broader independent workloads and paired downstream evidence showing that total successful-task cost falls without meaningful quality loss.

References

1 Huiqiang Jiang, Qianhui Wu, Chin-Yew Lin, Yuqing Yang, and Lili Qiu. "LLMLingua: Compressing Prompts for Accelerated Inference of Large Language Models." EMNLP 2023. DOI: 10.18653/v1/2023.emnlp-main.825. https://aclanthology.org/2023.emnlp-main.825/

2 Huiqiang Jiang, Qianhui Wu, Xufang Luo, Dongsheng Li, Chin-Yew Lin, Yuqing Yang, and Lili Qiu. "LongLLMLingua: Accelerating and Enhancing LLMs in Long Context Scenarios via Prompt Compression." ACL 2024. DOI: 10.18653/v1/2024.acl-long.91. https://aclanthology.org/2024.acl-long.91/

3 Zhuoshi Pan et al. "LLMLingua-2: Data Distillation for Efficient and Faithful Task-Agnostic Prompt Compression." Findings of ACL 2024. DOI: 10.18653/v1/2024.findings-acl.57. https://aclanthology.org/2024.findings-acl.57/

4 Yucheng Li, Bo Dong, Frank Guerin, and Chenghua Lin. "Compressing Context to Enhance Inference Efficiency of Large Language Models." EMNLP 2023. DOI: 10.18653/v1/2023.emnlp-main.391. https://aclanthology.org/2023.emnlp-main.391/

5 Fangyuan Xu, Weijia Shi, and Eunsol Choi. "RECOMP: Improving Retrieval-Augmented LMs with Compression and Selective Augmentation." ICLR

6 https://proceedings.iclr.cc/paper_files/paper/2024/file/bda88ed2892f5e61c9a9bf215c566913-Paper-Conference.pdf

7 Jinyi Li, Yihuai Lan, Lei Wang, and Hao Wang. "PCToolkit: A Unified Plug-and-Play Prompt Compression Toolkit of Large Language Models." arXiv:2403.17411, 2024. https://arxiv.org/abs/2403.17411

8 Nelson F. Liu et al. "Lost in the Middle: How Language Models Use Long Contexts." TACL 12, 2024. DOI: 10.1162/tacl_a_00638. https://aclanthology.org/2024.tacl-1.9/

9 Rico Sennrich, Barry Haddow, and Alexandra Birch. "Neural Machine Translation of Rare Words with Subword Units." ACL 2016. DOI: 10.18653/v1/P16-1162. https://aclanthology.org/P16-1162/

10 Taku Kudo and John Richardson. "SentencePiece: A Simple and Language Independent Subword Tokenizer and Detokenizer for Neural Text Processing." EMNLP System Demonstrations 2018. DOI: 10.18653/v1/D18-2012. https://aclanthology.org/D18-2012/

11 OpenAI. `tiktoken`, official BPE tokenizer implementation, release 0.12.0. https://github.com/openai/tiktoken/releases/tag/0.12.0

12 Anders Rundgren, Bret Jordan, and Samuel Erdtman. "JSON Canonicalization Scheme." RFC 8785, 2020. https://www.rfc-editor.org/rfc/rfc8785.html

13 NIST. "Secure Hash Standard." FIPS PUB 180-4, 2015. DOI: 10.6028/NIST.FIPS.180-4. https://nvlpubs.nist.gov/nistpubs/FIPS/NIST.FIPS.180-4.pdf

14 Koen Claessen and John Hughes. "QuickCheck: A Lightweight Tool for Random Testing of Haskell Programs." ICFP 2000. DOI: 10.1145/351240.351266. https://doi.org/10.1145/351240.351266

15 Jacob Ziv and Abraham Lempel. "A Universal Algorithm for Sequential Data Compression." IEEE Transactions on Information Theory 23(3), 1977. DOI: 10.1109/TIT.1977.1055714. https://www.itsoc.org/publications/papers/a-universal-algorithm-for-sequential-data-compression

16 Microsoft. LLMLingua v0.2.2, commit `a411a3fa61df74411157b2512b592d5357bd8f17`, MIT. https://github.com/microsoft/LLMLingua/tree/a411a3fa61df74411157b2512b592d5357bd8f17

17 Headroom Labs. Headroom v0.34.0, commit `9fd5ae3d530b5d9eb9e9127e413235a5a3b2faf6`, Apache-2.0. https://github.com/headroomlabs-ai/headroom/tree/9fd5ae3d530b5d9eb9e9127e413235a5a3b2faf6

18 RTK AI. RTK v0.45.0, commit `b34be37caf3796b69a50952a28e60e32b5daad43`, Apache-2.0. https://github.com/rtk-ai/rtk/tree/b34be37caf3796b69a50952a28e60e32b5daad43

19 Julius Brussee. Caveman v1.10.0, commit `fcf7663366c217dc8f334a11028de52ed950ceab`, MIT. https://github.com/JuliusBrussee/caveman/tree/fcf7663366c217dc8f334a11028de52ed950ceab

20 Diego Souza et al. OmniRoute v3.8.50, commit `1e15583f294d9d137320fe544288ca34c9435351`, MIT. The benchmark invokes only pinned pure compression modules. https://github.com/diegosouzapw/OmniRoute/tree/1e15583f294d9d137320fe544288ca34c9435351

21 OpenAI, Anthropic, and Google. Prompt/context caching documentation, accessed 2026-08-08. https://developers.openai.com/api/docs/guides/prompt-caching, https://platform.claude.com/docs/en/build-with-claude/prompt-caching, and https://ai.google.dev/gemini-api/docs/caching.

Appendix A. Minimal request example

```
{
  "schema_version": "kendr.optimize/v1",
  "phase": "request",
  "request_id": "paper-demo-001",
  "content": {
    "messages": [
      {
        "id": "u1",
        "role": "user",
        "parts": [
          {
            "type": "text",
            "text": "Diagnose the build failure. Do not change the API."
          }
        ]
      }
    ],
    "tools": []
  },
  "target": {
    "tokenizer_profile": "o200k_base",
    "model": "optional accounting hint"
  },
  "generation": {
    "expected_output_tokens": 800,
    "requested_verbosity": "auto"
  },
  "host_capabilities": {
    "can_narrow_tools": false,
    "can_restore_references": false,
    "can_retry_with_full_tools": false,
    "streaming_output": true,
    "can_set_verbosity": true
  },
  "policy": {
    "risk_ceiling": "recoverable",
    "min_gain_tokens": 8,
    "min_gain_percent": 1,
    "latency_budget_ms": 25,
    "preserve_cache_prefix": true,
    "enable_generation_policy": false
  }
}
```

Appendix B. Reviewer checklist for a new engine

- Is the engine independently implemented and provenance documented?
- Are accepted phases, roles, and content types narrow and explicit?
- Is its risk class no lower than the actual behavioral risk?
- Does it reject its own marker or encoding to prevent double application?
- Are parser and expansion limits checked before allocation?
- Are tool calls, exact typed parts, output contracts, and required tools unchanged?
- Are protected artifacts preserved with multiplicity?
- Can every recoverable marker expand independently to the precise prior envelope?
- Are cache-bound messages untouched unless policy permits it?
- Is gain measured after complete envelope serialization with signed deltas?
- Are no-op, rejection, and revert reasons visible in the receipt?
- Do golden, adversarial, malformed-input, round-trip, and fuzz tests exist?
- Does the benchmark keep failures and no-ops in coverage and qualification accounting, and include all consumed usage in paired workflow cost?
- Is any downstream-quality claim supported by paired task-native evidence?

Appendix C. Reading a ranking row

A ranking row should answer all of these questions without requiring marketing interpretation:

1. Which content surface was transformed?
2. Which exact version and setting ran?
3. Which cases were eligible, completed, no-op, or failed?
4. Which tokenizer independently recounted the payload?
5. What was the signed raw reduction?
6. Did every completed case pass its composite fixture-preservation gate?
7. What risk class did the transformation use?
8. Was the target model executed?
9. Was provider usage paired with a baseline?
10. Was downstream quality scored under a preregistered criterion?

If any of these fields is unavailable, the row should say `N/A` or **`not measured`**; it should not infer success from a large token percentage.